

## Executive Summary

This project investigates color-based image segmentation using a manually implemented K-Means clustering pipeline. Rather than calling a ready-made K-Means model, the notebook builds the core algorithm step by step: K-Means++ centroid initialization, nearest-centroid assignment, centroid updating, empty-cluster handling, convergence checking, repeated initialization, and final image reconstruction. The primary landscape image was resized from  $4928 \times 3264$  to  $800 \times 530$ , lightly smoothed, converted into normalized RGB pixel vectors, and segmented into three dominant color regions after an elbow-and-silhouette analysis. The final primary segmentation converged in 14 iterations with an inertia per pixel of 0.012294, a sampled silhouette score of 0.6924, PSNR of 23.8736 dB, and SSIM of 0.7555. The image color space was reduced from 82,773 unique colors to three representative colors, corresponding to an exact reduction of approximately 99.9964%.

Two additional images - a colorful fruit scene and an urban street scene - were also processed to test whether the same implementation behaves consistently on visually different content. Both runs converged. The fruit image achieved the strongest reconstruction similarity (PSNR 24.7580 dB and SSIM 0.8686), while the primary landscape produced the highest silhouette score (0.6924), indicating the clearest sampled cluster separation among the three experiments. The results show that the implementation is stable and useful for color quantization and unsupervised region grouping, while also revealing an important limitation: reconstruction metrics such as PSNR and SSIM are not the same as ground-truth segmentation accuracy.

## 1. Introduction

Image segmentation divides an image into meaningful regions so that areas with similar visual properties can be studied separately. In this project, similarity is defined in RGB color space. Each pixel is treated as a three-dimensional feature vector, and K-Means clustering groups pixels that are close to one another in color. The final centroid of each cluster becomes a representative color for that region.

The main purpose of the work is not only to obtain a segmented image, but also to expose the complete clustering process. The notebook therefore implements the algorithm manually and validates each stage before running the final full-image experiment. This makes the workflow suitable for a Digital Image Processing course because the important computational steps remain visible and auditable instead of being hidden inside a library model.

## 2. Project Objectives

- Load and validate an RGB image and prepare it for efficient processing.
- Study the RGB channels and intensity distributions of the image.
- Represent every image pixel as a normalized RGB feature vector.
- Implement K-Means++ initialization, pixel assignment, centroid updating, convergence testing, and empty-cluster recovery from scratch.
- Evaluate several candidate K values using inertia and silhouette analysis before selecting the final number of color regions.
- Reconstruct a segmented image from cluster centroids and generate label maps, binary masks, extracted regions, and boundary overlays.
- Measure clustering and reconstruction quality using inertia, silhouette score, MSE, RMSE, MAE, PSNR, SSIM, color reduction, and an estimated palette-compression ratio.
- Test the reusable segmentation pipeline on two additional images and compare the behavior across different scenes.

### 3. Experimental Setup

Table 1. Main project configuration

| Parameter                        | Value               |
|----------------------------------|---------------------|
| Random state                     | 42                  |
| Maximum image side               | 800 pixels          |
| Gaussian smoothing               | Enabled, radius = 1 |
| Candidate K values               | 2, 3, 4, 5, 6, 7, 8 |
| Initial implementation-test K    | 5                   |
| Maximum K-Means iterations       | 50                  |
| Convergence tolerance            | $1 \times 10^{-4}$  |
| K-selection pixel sample         | 20,000 pixels       |
| Silhouette sample                | 3,000 pixels        |
| Assignment chunk size            | 100,000 pixels      |
| K-selection initializations      | 3 per K             |
| Final full-image initializations | 5                   |

The implementation uses NumPy and Pandas for numerical work and tabulation, Pillow for image loading and preprocessing, Matplotlib for visualization, scikit-learn only for the silhouette metric, scikit-image for SSIM, and SciPy for a small boundary-dilation step. The actual K-Means clustering itself is manually implemented in the notebook.

### 4. Methodology

#### 4.1 Image Loading, Validation, and Preprocessing

The primary JPEG image had a resolution of  $4928 \times 3264$  (16,084,992 pixels). To keep the clustering computation practical while preserving the scene geometry, the longest side was reduced to 800 pixels using Lanczos resampling. The resulting processing resolution was  $800 \times 530$ , or 424,000 pixels, which reduced the number of pixels by 97.36%. A light Gaussian blur with radius 1 was then applied to reduce isolated color noise before clustering.



Figure 1. Original, resized, and lightly smoothed versions of the primary landscape image.

Table 2. Primary image preprocessing summary

| Item                   | Result                  |
|------------------------|-------------------------|
| Original resolution    | 4928 × 3264             |
| Processing resolution  | 800 × 530               |
| Original pixel count   | 16,084,992              |
| Processing pixel count | 424,000                 |
| Pixel-count reduction  | 97.36%                  |
| Resize scale           | 0.1623                  |
| Color channels         | 3 (RGB)                 |
| Smoothing              | Gaussian blur, radius 1 |

4.2 RGB Analysis and Feature Construction

The processed image was separated into red, green, and blue channels. The channel statistics show that the green channel had the highest mean intensity (171.31), while the blue channel had the largest standard deviation (75.54), indicating the widest intensity spread. The histogram provides a visual view of these color distributions before clustering.

Table 3. RGB channel statistics

| Channel | Minimum | Maximum | Mean   | Std. Dev. |
|---------|---------|---------|--------|-----------|
| Red     | 38      | 220     | 148.69 | 51.34     |
| Green   | 42      | 222     | 171.31 | 43.94     |
| Blue    | 14      | 217     | 129.47 | 75.54     |

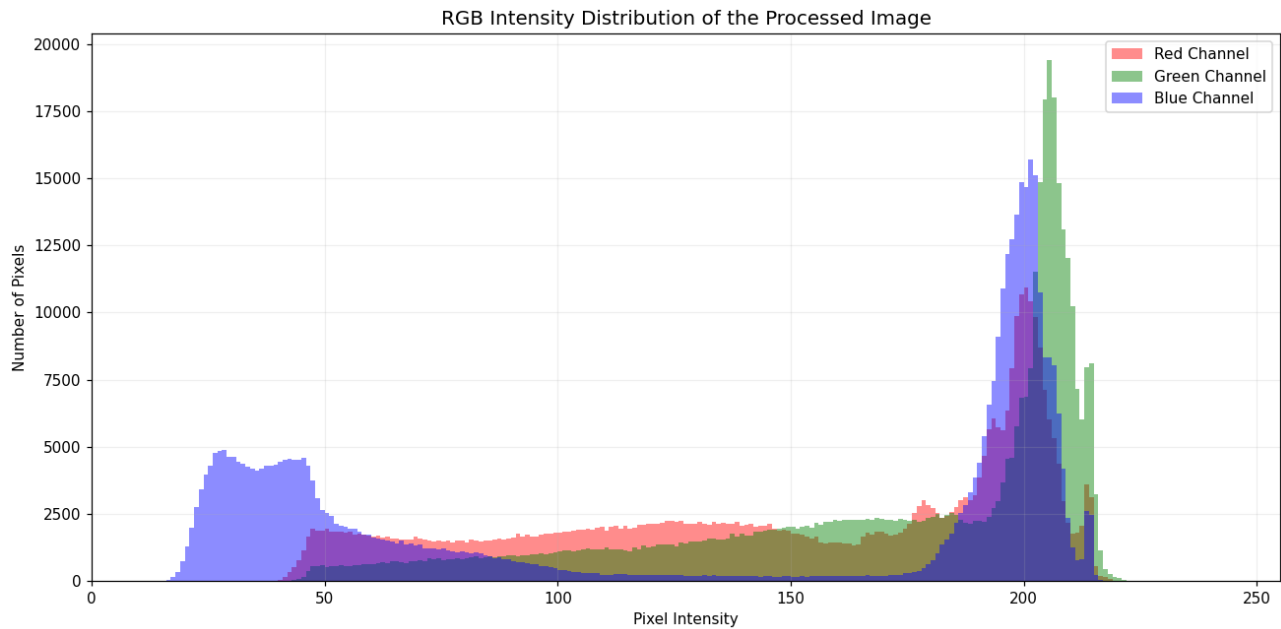


Figure 2. RGB intensity distribution of the processed primary image.

For clustering, the  $800 \times 530 \times 3$  image was reshaped into a  $424,000 \times 3$  feature matrix. RGB values were converted from integers in  $[0, 255]$  to floating-point values in  $[0, 1]$ . A reproducible 20,000-pixel sample was

drawn for K selection so that repeated candidate-K experiments remained efficient. The complete feature matrix occupied approximately 4.85 MB.

### 4.3 Manual K-Means++ Clustering

The clustering routine was built from separate functions so that each part could be tested independently. K-Means++ was used to initialize centroids: the first centroid was selected randomly, and later centroids were sampled with probability proportional to the squared distance from the nearest existing centroid. Pixel assignment used squared Euclidean distance, processed in chunks of 100,000 pixels to avoid creating an unnecessarily large distance matrix. New centroids were computed as the mean RGB vector of all pixels assigned to each cluster. If a cluster became empty, the implementation reinitialized it using a high-error pixel that was far from its current assigned centroid.

Iterations continued until the largest centroid movement became less than or equal to 0.0001 or the 50-iteration limit was reached. A preliminary K = 5 run on the 20,000-pixel sample was used as an implementation check. It converged in 20 iterations, reducing inertia from 445.996571 to 131.845622 - a 70.44% reduction - while the final centroid shift fell to approximately 0.000050, below the convergence tolerance.

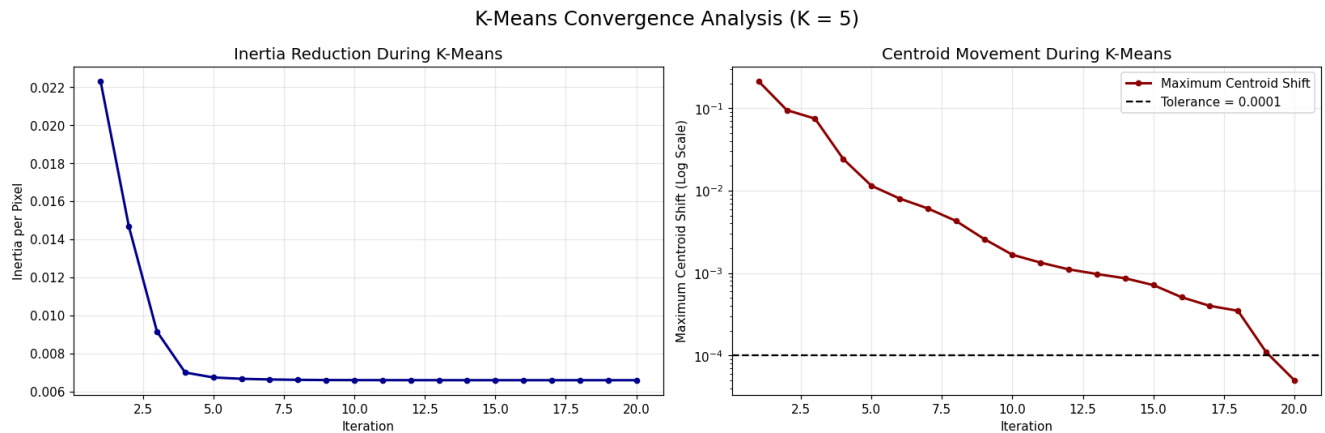


Figure 3. Convergence behavior of the manual K-Means implementation during the sample validation run (K = 5).

### 4.4 Selecting the Number of Color Clusters

Candidate values from K = 2 to K = 8 were evaluated on the 20,000-pixel sample. Each K was run three times with different seeds, and the lowest-inertia run was retained. Inertia decreased steadily as K increased, while the silhouette score was highest at K = 2. An automatic elbow detector based on the maximum perpendicular distance from the line connecting the first and last normalized inertia points identified K = 3. The notebook selected K = 3 because it provided a practical balance between color separation and retained image detail.

Table 4. Candidate K evaluation

| K | Inertia / Pixel | Silhouette | Iterations | Converged | Best Seed |
|---|-----------------|------------|------------|-----------|-----------|
| 2 | 0.025411        | 0.7538     | 5          | True      | 243       |
| 3 | 0.012409        | 0.6924     | 13         | True      | 342       |
| 4 | 0.008898        | 0.6544     | 17         | True      | 444       |
| 5 | 0.006592        | 0.5361     | 22         | True      | 542       |
| 6 | 0.005284        | 0.5270     | 43         | True      | 642       |
| 7 | 0.004325        | 0.5350     | 50         | True      | 743       |
| 8 | 0.003760        | 0.5225     | 34         | True      | 842       |

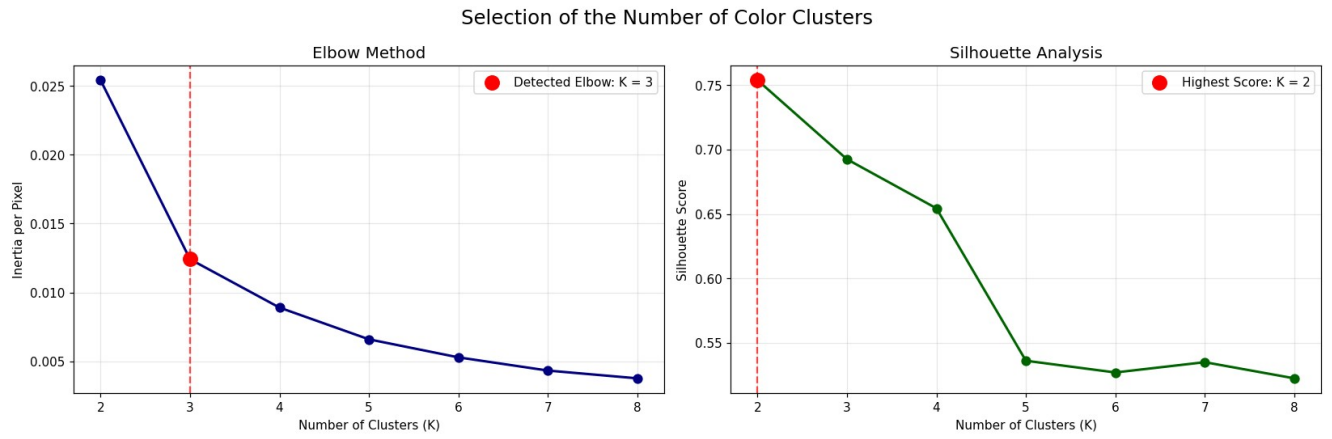


Figure 4. Elbow and silhouette analyses used for selecting the final number of color clusters.

#### 4.5 Final Full-Image Segmentation and Reconstruction

After  $K = 3$  was selected, the complete set of 424,000 pixels was clustered five times using seeds 1042-1046. All five runs converged and produced nearly identical final inertia values. The lowest-inertia run was initialization 1 (seed 1042), which converged in 14 iterations. Its centroid colors were then mapped back to the image grid so that each pixel was replaced by the representative RGB color of its assigned cluster.

Table 5. Final full-image initialization comparison

| Run | Seed | Inertia   | Inertia / Pixel | Iterations | Converged | Runtime (s) |
|-----|------|-----------|-----------------|------------|-----------|-------------|
| 1   | 1042 | 5212.7632 | 0.012294        | 14         | True      | 1.7324      |
| 2   | 1043 | 5212.7635 | 0.012294        | 12         | True      | 1.0161      |
| 3   | 1044 | 5212.7639 | 0.012294        | 14         | True      | 1.1763      |
| 4   | 1045 | 5212.7638 | 0.012294        | 11         | True      | 0.9454      |
| 5   | 1046 | 5212.7633 | 0.012294        | 11         | True      | 0.9300      |



Figure 5. Primary-image segmentation result: original resized image, preprocessed image, and K-Means output for  $K = 3$ .

#### 4.6 Dominant Colors, Region Masks, and Boundaries

The final cluster centroids were converted back to 8-bit RGB values and ranked by the number of assigned pixels. The largest region was a light gray-green color and covered 54.17% of the image. The remaining two green-toned clusters covered 25.77% and 20.06%, respectively. Together the three masks covered every pixel exactly once, confirming that the cluster label map was complete and non-overlapping.

Table 6. Dominant color distribution of the primary image

| Rank | Cluster   | RGB             | Hex     | Pixels  | Share  |
|------|-----------|-----------------|---------|---------|--------|
| 1    | Cluster 3 | (190, 204, 197) | #BECC5  | 229,666 | 54.17% |
| 2    | Cluster 1 | (124, 161, 58)  | #7CA13A | 109,283 | 25.77% |
| 3    | Cluster 2 | (69, 97, 39)    | #456127 | 85,051  | 20.06% |

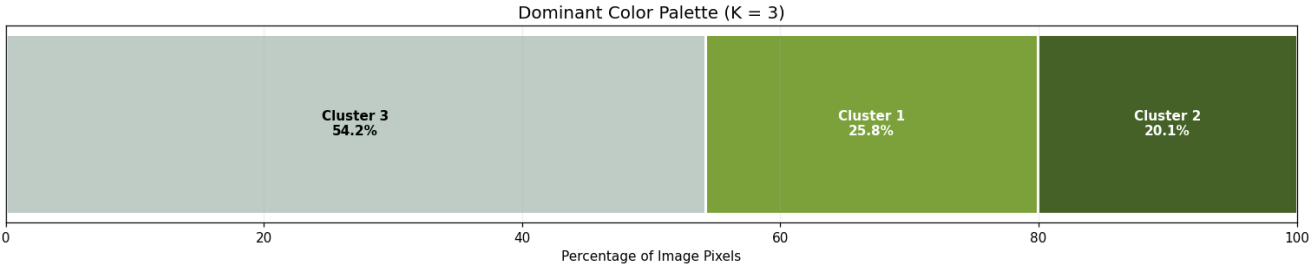


Figure 6. Proportional dominant-color palette for the primary segmentation.

Binary masks were generated from the cluster label map, and the original processed pixels belonging to each cluster were extracted separately. Boundaries were detected wherever horizontally or vertically adjacent pixels had different cluster labels. A one-iteration binary dilation made the boundaries easier to see, and the boundary pixels were overlaid in red. The visible boundary mask contained 73,041 pixels, equivalent to 17.23% of the processed image.

## K-Means Segmentation Boundary Analysis

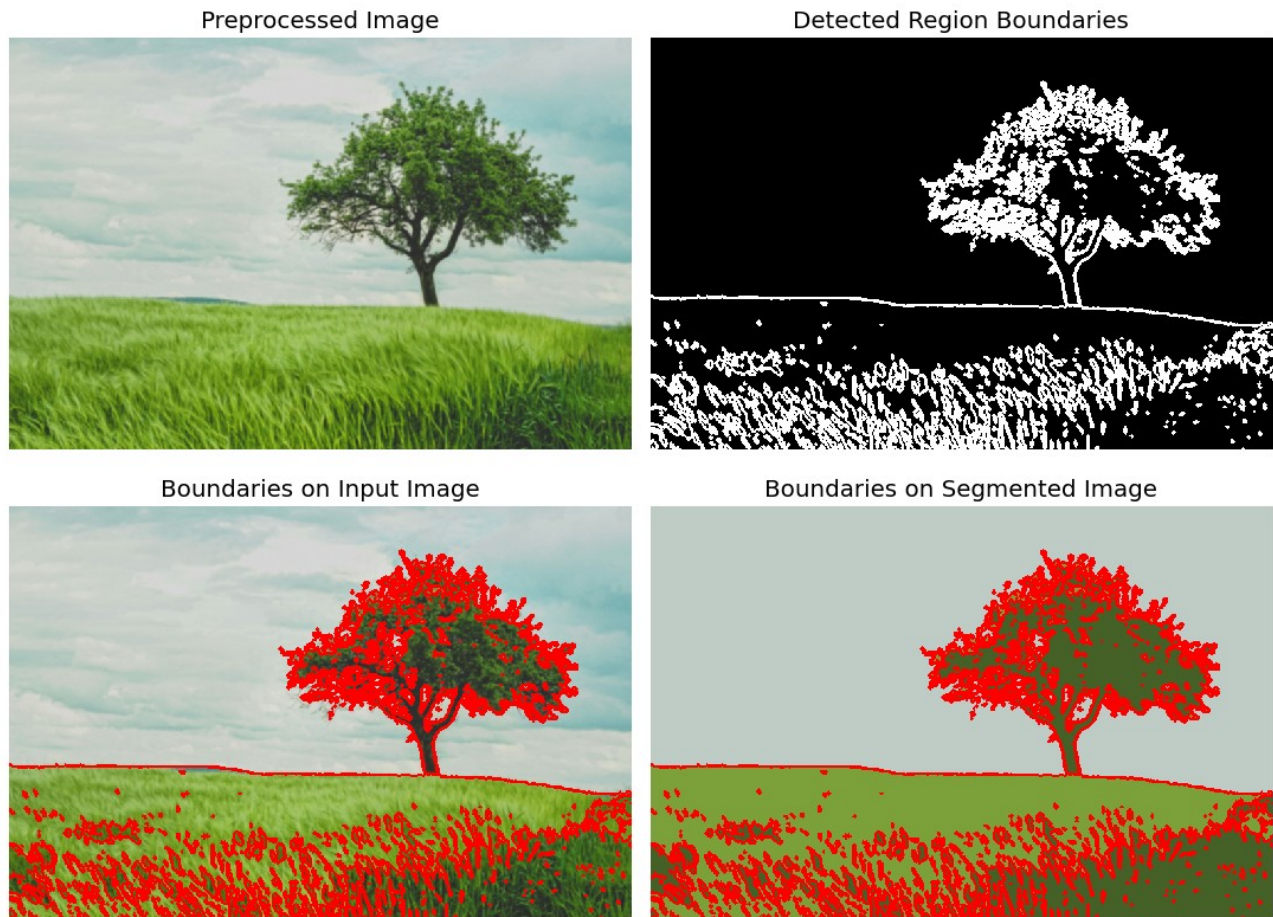


Figure 7. Region-boundary detection and red overlays on the input and segmented images.

## 5. Results and Discussion

### 5.1 Primary Image Quantitative Results

The final segmentation produced a compact three-color representation while preserving the large-scale structure of the scene. The sampled silhouette score of 0.6924 indicates reasonably well-separated color clusters. Reconstruction quality remained moderate: PSNR was 23.8736 dB and SSIM was 0.7555. These values are expected to decrease as an image is aggressively quantized to only three colors because fine color variation is intentionally discarded.

**Table 7. Final quantitative evaluation of the primary segmentation**

| Metric            | Value    | Interpretation                          |
|-------------------|----------|-----------------------------------------|
| Selected K        | 3        | Three dominant color regions            |
| Converged         | True     | Stopping criterion was reached          |
| Iterations        | 14       | Iterations in the selected final run    |
| Inertia per pixel | 0.012294 | Average within-cluster squared distance |
| Silhouette score  | 0.6924   | Sampled cluster separation/compactness  |
| MSE               | 266.5119 | Mean squared reconstruction error       |

| Metric                  | Value          | Interpretation                               |
|-------------------------|----------------|----------------------------------------------|
| RMSE                    | 16.3252        | Root mean squared reconstruction error       |
| MAE                     | 11.9870        | Mean absolute reconstruction error           |
| PSNR                    | 23.8736 dB     | Color-quantization reconstruction quality    |
| SSIM                    | 0.7555         | Structural similarity to the processed input |
| Original unique colors  | 82,773         | Unique colors before segmentation            |
| Segmented unique colors | 3              | Unique colors after segmentation             |
| Color reduction         | 99.9964% exact | Rounded to 100.00% in notebook summary       |
| Boundary percentage     | 17.23%         | Visible dilated boundary pixels              |
| Theoretical compression | 12.00:1        | Palette-index estimate, not file compression |

Image Segmentation using Manual K-Means Clustering  
K = 3 | Iterations = 14 | Silhouette = 0.6924 | PSNR = 23.87 dB | SSIM = 0.7555



Figure 8. Final project dashboard summarizing the primary-image segmentation.

The exact color reduction is approximately 99.9964%, even though the report table generated inside the notebook rounds this to 100.00%. This distinction matters because three colors remain after segmentation. Likewise, the 12:1 value is a theoretical palette-index representation estimate; it should not be interpreted as the compression ratio of a JPEG or PNG file on disk.

## 5.2 Additional Image Testing

To check whether the implementation was reusable beyond the primary scene, the notebook processed two additional images. Both were resized to  $800 \times 534$  and segmented with  $K = 6$  using three initializations. The fruit image converged in 19 iterations, while the more visually complex urban scene required 40 iterations. Both still met the convergence condition.

Table 8. Additional test-image results

| Image           | K | Pixels  | Best Run | Iterations | Inertia / Pixel | Silhouette | PSNR (dB) | SSIM   |
|-----------------|---|---------|----------|------------|-----------------|------------|-----------|--------|
| Colorful Fruits | 6 | 427,200 | 3        | 19         | 0.010025        | 0.5003     | 24.7580   | 0.8686 |
| Urban Street    | 6 | 427,200 | 1        | 40         | 0.012879        | 0.4634     | 23.6705   | 0.7156 |

### Additional Image Testing using Manual K-Means

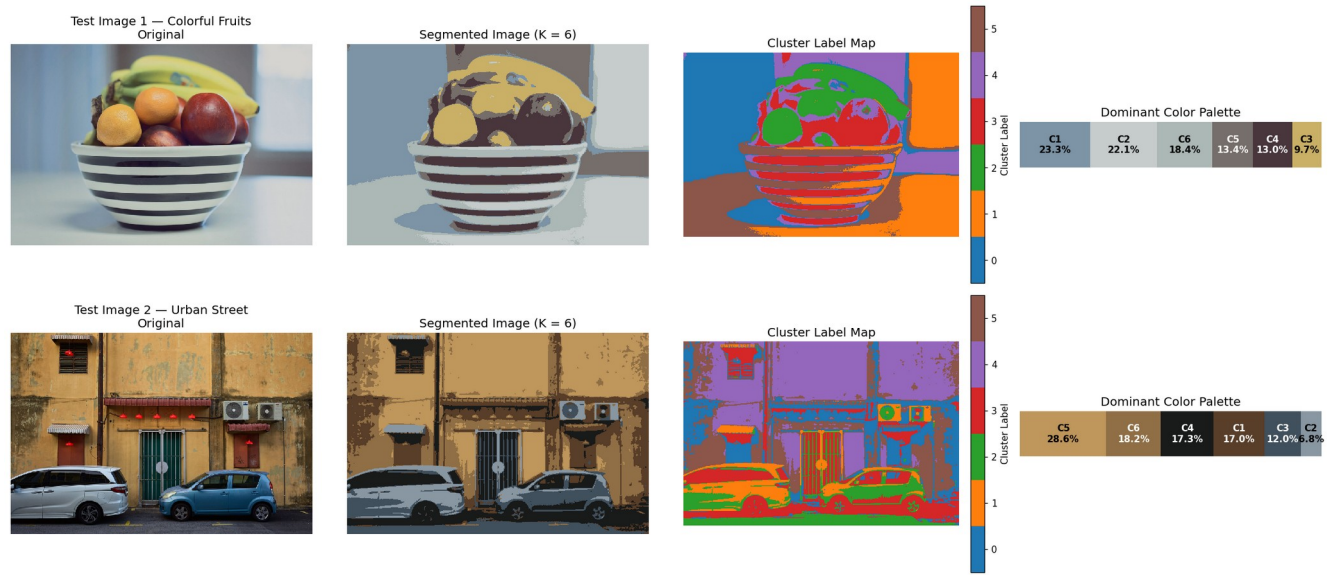


Figure 9. Additional-image results showing original images, segmented outputs, label maps, and dominant-color palettes.

## 5.3 Cross-Image Comparison

Table 9. Comparison across all three experiments

| Image             | K | Iterations | Converged | Inertia / Pixel | Silhouette | PSNR (dB) | SSIM   | Runtime (s) |
|-------------------|---|------------|-----------|-----------------|------------|-----------|--------|-------------|
| Primary Landscape | 3 | 14         | True      | 0.012294        | 0.6924     | 23.8736   | 0.7555 | 1.7324      |
| Colorful Fruits   | 6 | 19         | True      | 0.010025        | 0.5003     | 24.7580   | 0.8686 | 2.5140      |
| Urban Street      | 6 | 40         | True      | 0.012879        | 0.4634     | 23.6705   | 0.7156 | 5.2381      |

## Performance Comparison Across Three Images

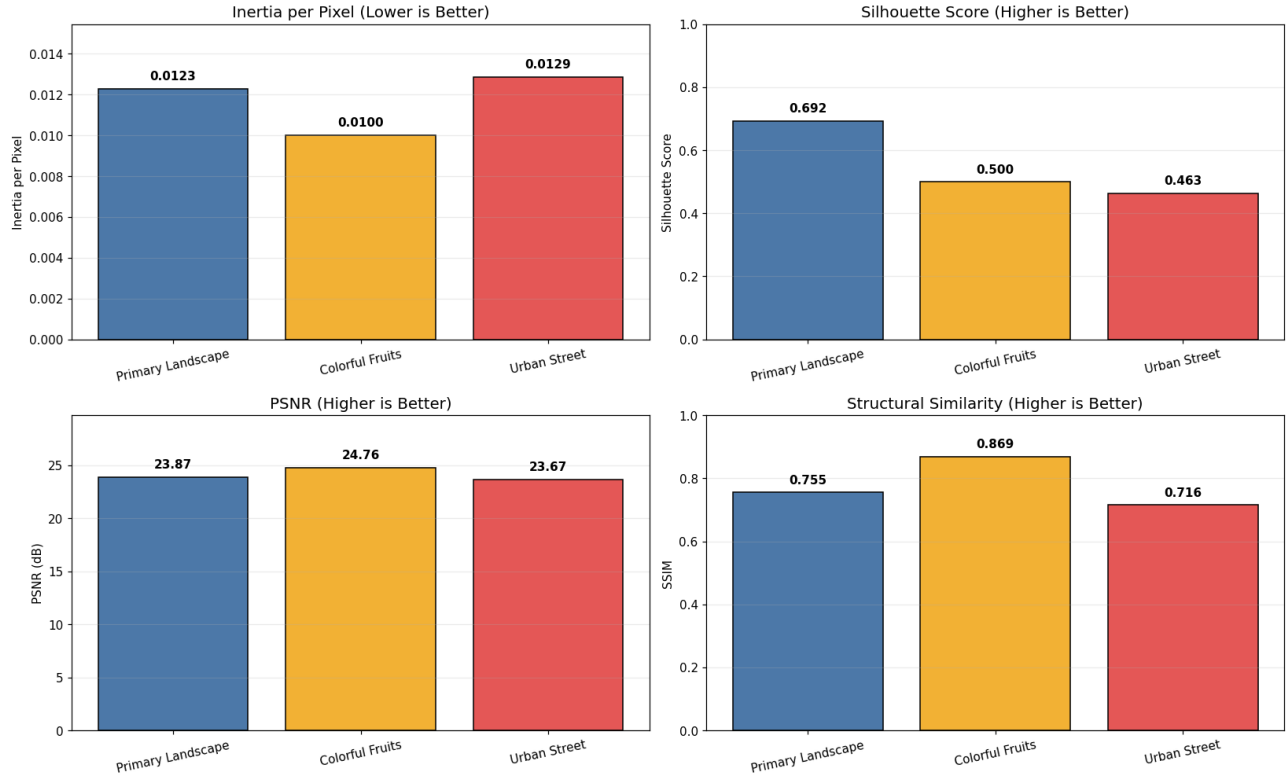


Figure 10. Performance comparison across the primary landscape, colorful fruits, and urban street images.

The fruit image achieved the lowest inertia per pixel, highest PSNR, and highest SSIM of the three experiments, indicating the strongest reconstruction agreement after color quantization. The primary landscape had the highest silhouette score, suggesting the clearest sampled separation between its selected color groups. The urban street was the most computationally demanding of the three, requiring 40 iterations and the longest selected-run time (5.2381 s).

This cross-image comparison should be interpreted carefully because the primary image used  $K = 3$  while both additional images used  $K = 6$ . Increasing  $K$  normally gives the segmented image more representative colors, which can improve reconstruction metrics. Therefore, the table is best read as a robustness check of the implementation across different scenes, not as a strict benchmark where every experimental condition is identical.

## 6. Validation and Reliability Checks

- Input validation ensured that uploaded files were valid images and converted them to RGB.
- Feature-matrix shape, finite values, and pixel counts were checked before clustering.
- The assignment function verified that every pixel received a cluster label.
- The centroid-update stage explicitly handled the possibility of empty clusters.
- Convergence history was recorded, and inertia was checked to be non-increasing during the sample validation run.
- Multiple random initializations were used for candidate  $K$  evaluation, final full-image clustering, and additional image testing.
- The final mask set was checked to cover all pixels exactly once.
- All three reported experiments converged successfully.

## 7. Limitations

The project is a strong color-clustering study, but several limitations should be stated clearly so that the results are not overinterpreted:

- K-Means groups pixels only by RGB color. It does not use spatial coordinates, texture, edges, or semantic object information, so similarly colored but unrelated areas can be assigned to the same cluster.
- The primary silhouette score is computed on a controlled sample rather than on all 424,000 pixels. This makes evaluation efficient, but the value is an approximation of full-image cluster separation.
- PSNR and SSIM compare the segmented image with the processed input. They measure reconstruction similarity after color quantization; they are not ground-truth segmentation-accuracy metrics such as Dice or IoU.
- The additional images use  $K = 6$  by configuration rather than repeating the full K-selection procedure. Their metrics are therefore useful for robustness testing but are not directly comparable under identical model complexity.
- The 12:1 compression figure is theoretical and assumes palette/index storage. It does not represent the compressed size of an actual PNG or JPEG output.
- The primary landscape image source URL was not stored in the notebook. It should be added before final academic submission if the image was obtained from an external source.

## 8. Reproducibility and Saved Outputs

The notebook fixes the main random state at 42 and saves both numerical results and visual artifacts. The primary experiment exports 20 files, including the resized/preprocessed/segmented images, boundary masks and overlays, dominant-color palette, label map, per-cluster masks and extracted regions, candidate-K results, initialization comparison, convergence history, final metrics, and a combined dashboard. A final project archive also groups the primary experiment, both additional tests, cross-image comparison files, and image-source information.

The final archive reported by the notebook was named `kmeans_complete_project_results.zip` and had a size of approximately 8.41 MB. This output organization supports later review because the numerical tables and figures can be inspected without rerunning every notebook cell.

## 9. Conclusion

The project successfully demonstrates a complete color-based image segmentation workflow using a manual K-Means++ implementation. The code does more than produce a final picture: it validates the data, shows how centroids are initialized and updated, checks convergence, evaluates candidate cluster counts, reconstructs the segmented image, isolates individual regions, visualizes boundaries, and reports quantitative metrics. The primary landscape was reduced to three dominant color regions and converged consistently across five final initializations, while two additional images confirmed that the same implementation can handle different visual scenes.

The results are strongest when interpreted as unsupervised color segmentation and color quantization rather than semantic object segmentation. Within that scope, the workflow is technically sound, reproducible, and well suited to a Digital Image Processing project because the algorithmic steps remain transparent from input image to final evaluation.

## 10. Image Sources

Table 10. Image-source information recorded in the notebook

| Image             | Source                                                                                                                                                                                    |
|-------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Primary Landscape | User-selected image. Original source URL was not stored in the notebook and should be added before submission if applicable.                                                              |
| Colorful Fruits   | Pexels: <a href="https://www.pexels.com/photo/sliced-mixed-fruits-in-a-white-ceramic-bowl-8805091/">https://www.pexels.com/photo/sliced-mixed-fruits-in-a-white-ceramic-bowl-8805091/</a> |
| Urban Street      | Pexels: <a href="https://www.pexels.com/photo/urban-city-street-with-colorful-buildings-34800961/">https://www.pexels.com/photo/urban-city-street-with-colorful-buildings-34800961/</a>   |

## Appendix A. Notebook Workflow

The uploaded notebook contains 26 executable code cells. Its sequence is: environment setup; visualization utility; primary image upload; preprocessing; RGB analysis; feature construction; manual K-Means++ initialization; assignment; centroid update; full iterative K-Means; convergence verification; candidate-K evaluation; K selection; final full-image clustering; reconstruction; dominant-color analysis; masks/region extraction; boundary analysis; quantitative evaluation; final dashboard; primary-result export; reusable test function; two-image testing; additional-image dashboard; cross-image comparison; and final result packaging.